

Neural Network and Deep Learning — Project 1

姓名：于翔 学号：23307130202

2026 年 5 月 11 日

摘要

本项目研究 MNIST 手写数字分类任务。首先基于 NumPy 从零实现 MLP baseline，包括线性层前向/反向传播、ReLU、softmax cross-entropy 和 SGD 训练；随后手写二维卷积算子，构建两层 CNN 并与 MLP 公平比较。完成基础部分后，选择 **optimization** 与 **regularization** 两个方向做控制变量实验。实验显示：MLP test accuracy 0.9544；手写 CNN 提升到 0.9779（参数量反而更少）；最佳训练 recipe 达 0.9856。所有算子均通过数值梯度检验（相对误差 $< 10^{-8}$ ）。

1 Experimental Setup

1.1 Dataset and Split

MNIST 包含 60 000 张训练图像和 10 000 张测试图像，每张 28×28 灰度图，标签 $y \in \{0, \dots, 9\}$ ，像素归一化到 $[0, 1]$ 。固定种子 233，划分 10 000 张为 validation set，50 000 张为 training set；测试集仅用于最终评估。

1.2 Code Organization and Gradient Verification

主要实现位于 codes/ 目录：mynn/op.py (Linear, conv2D, ReLU, MaxPool2D, Dropout, CE loss), mynn/models.py (Model_MLP, Model_CNN), mynn/optimizer.py (SGD, Momentum, Adam 等), mynn/lr_scheduler.py (StepLR, MultiStepLR, ExponentialLR), mynn/runner.py (训练循环)，以及训练脚本 test_train.py、Part C 实验脚本 experiment_part_c_recipe.py 和分析脚本 analysis_visualization.py。

为验证实现正确性，使用中心差分法 ($\epsilon = 10^{-5}$) 对所有核心算子做数值梯度检验：

Operator	w.r.t.	Rel. error	Operator	w.r.t.	Rel. error
Linear	W	7.4×10^{-11}	conv2D	W	3.2×10^{-11}
Linear	b	1.9×10^{-11}	conv2D	X	2.6×10^{-10}
Linear	X	2.5×10^{-9}	MaxPool2D	X	1.1×10^{-10}
Softmax CE	logits	2.4×10^{-9}			

表 1: Numerical gradient check. All relative errors $< 10^{-8}$.

1.3 Reproducibility and Links

```
git clone https://github.com/Parry-Git/Deep-Learning-Project1.git
cd Deep-Learning-Project1
conda env create -f environment.yml && conda activate dl-pj1
python scripts/download_checkpoints.py # download saved models
python scripts/verify_submission.py    # verify accuracy
python codes/gradient_check.py         # gradient check
```

Known issues:

- **Git LFS.** 模型 checkpoint 以 Git LFS 存储在 ModelScope。若未安装 git-lfs, git clone 只会下载 pointer 文件 (~131 B), 导致 pickle.load 报错 invalid load key, 'v'。解决方法:

```
brew install git-lfs # macOS; Linux: apt install git-lfs
git lfs install
```

安装后重新运行 download_checkpoints.py 即可。

- **NumPy 版本兼容.** checkpoint 以 NumPy ≥ 2.0 序列化, 其内部模块路径为 numpy._core。若加载环境为 NumPy 1.x (路径为 numpy.core), 反序列化会抛出 ModuleNotFoundError。评估脚本已内置兼容 shim; 如遇此错误, 升级 NumPy 或确认使用 environment.yml 创建环境。

Links: Code: [GitHub](#) | Checkpoints: [ModelScope](#)

2 Part A: MLP Baseline

2.1 Architecture and Formulation

MLP 将 28×28 图像展平为 784-维向量, 经两层隐藏层输出 logits:

$$784 \xrightarrow{\text{Linear+ReLU}} 32 \xrightarrow{\text{Linear+ReLU}} 16 \xrightarrow{\text{Linear}} 10.$$

第 l 层前向传播: $Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}$, $A^{(l)} = \text{ReLU}(Z^{(l)})$ 。隐藏层维度选择 $32 \rightarrow 16$ 形成逐层压缩的瓶颈结构, 在参数量受限 (总计 25 818) 的条件下保留足够表达能力。输入维度 784 远大于隐藏维度, 第一层 784×32 占据绝大多数参数, 是 MLP 参数效率较低的主要来源。

权重初始化. 所有隐藏层权重采用 He 初始化: $W^{(l)} \sim \mathcal{N}(0, \sqrt{2/n_{\text{in}}})$, 其中 n_{in} 为该层输入维度。He 初始化针对 ReLU 激活函数的半区间特性 (期望约一半神经元输出为零) 设计, 使各层前向传播的方差保持稳定, 避免信号在深层指数衰减或爆炸。偏置项 $b^{(l)}$ 初始化为零。

ReLU 激活函数. $\text{ReLU}(x) = \max(0, x)$ 。相比 sigmoid/tanh, ReLU 在正半轴梯度恒为 1, 有效缓解深层网络中的梯度消失问题; 同时稀疏激活 (负值区域输出为零) 有助于学习更具判别性的特征。

Softmax cross-entropy. 为避免指数溢出, 计算时先减去 logits 最大值:

$$p_k = \frac{\exp(z_k - \max_j z_j)}{\sum_r \exp(z_r - \max_j z_j)}, \quad \ell = -\log p_y.$$

对 batch 中第 i 个样本的 logit $z_{i,k}$, 将 softmax 与 cross-entropy 合并求导可得到简洁的梯度形式: $\frac{\partial L}{\partial z_{i,k}} = \frac{p_{i,k} - \mathbb{I}[y_i=k]}{N}$, 其中 N 为 batch size。合并求导避免了分别计算 softmax 的 Jacobian 矩阵, 数值上也更加稳定。

Linear backward. 设上游梯度为 $G = \partial L / \partial Z^{(l)}$, 则各参数和输入的梯度为: $\partial L / \partial W = X^\top G$, $\partial L / \partial b = \sum_i G_i$, $\partial L / \partial X = GW^\top$ 。其中 $\partial L / \partial X$ 作为下一层反向传播的输入, 实现了链式法则的逐层回传。

ReLU backward. $\partial L / \partial Z = (\partial L / \partial A) \odot \mathbb{I}[Z > 0]$ 。ReLU 的梯度为分段常数: 正区间传递上游梯度, 负区间完全截断。前向传播时需缓存 Z 用于反向判断激活状态。

2.2 Training and Results

表 2: Part A configuration.

Optimizer / lr	SGD, $\eta=0.1$
Batch / Epochs	128 / 5
Init	He (ReLU layers)
Parameters	25 818

表 3: Part A results.

Metric	Value
Best val acc	0.9479
Test acc	0.9544

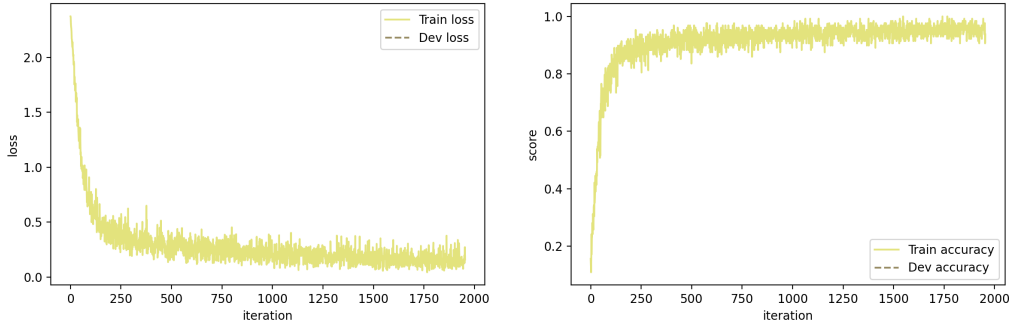


图 1: MLP baseline learning curve.

从学习曲线 (图 1) 可以观察到: 训练 loss 在前 2 个 epoch 内快速下降并逐步趋于平稳; 验证集 accuracy 约在 epoch 3 后开始收敛, 最终 test accuracy 达到 0.9544, 验证了所有算子实现的正确性。

MLP 的局限性. MLP 将二维图像展平为一维向量, 完全丢失了像素的空间邻域关系。这意味着: (1) 相邻像素间的局部相关性 (如笔画的连续性、边缘方向) 无法被结构性利用, 必须由全连接权重从数据中隐式学习; (2) 平移等价性 (同一数字出现在图像不同位置) 无法共享特征, 网络需为每个空间位置分别学习检测器, 显著增加所需参数量和训练样本量。这构成了引入卷积结构的核心动机。

3 Part B: CNN Model

3.1 Convolution and Pooling

输入 $X \in \mathbb{R}^{N \times C_{in} \times H \times W}$, 卷积核 $K \in \mathbb{R}^{C_{out} \times C_{in} \times R \times S}$ 。在 stride t 、padding p 下, 首先对输入做零填充得到 $\tilde{X}$ (四周各补 p 行/列零值), 输出空间尺寸为 $H_{out} = \lfloor (H + 2p - R)/t \rfloor + 1$, 前向计算为:

$$Y_{n,o,i,j} = b_o + \sum_{c,r,s} K_{o,c,r,s} \tilde{X}_{n,c,it+r,jt+s}.$$

实现上，前向传播使用 `numpy.lib.stride_tricks.sliding_window_view` 将输入展开为所有感受野窗口的视图（无数数据拷贝），再通过 `np.einsum` 完成卷积核与窗口的批量内积，避免显式四重循环，在纯 NumPy 下获得较好的计算效率。

Backward: 设上游梯度 $G = \partial L / \partial Y \in \mathbb{R}^{N \times C_{\text{out}} \times H_{\text{out}} \times W_{\text{out}}}$ ，卷积核梯度为各输出位置处输入窗口与对应上游梯度的相关： $\partial L / \partial K_{o,c,r,s} = \sum_{n,i,j} G_{n,o,i,j} \tilde{X}_{n,c,it+r,jt+s}$ ；偏置梯度为上游梯度沿 batch 和空间维度的求和： $\partial L / \partial b_o = \sum_{n,i,j} G_{n,o,i,j}$ 。输入梯度的计算需将上游梯度「散射」回对应的输入位置：遍历 kernel 的每个 (r, s) 偏移，将 G 与 K 的对应切片相乘后累加到 $\partial L / \partial \tilde{X}$ 的相应位置，最后去除 padding 区域的梯度。当 stride > 1 时需注意步长引起的索引间隔。

Max pooling: $Y_{n,c,i,j} = \max_{0 \leq r,s < P} X_{n,c,Pi+r,Pj+s}$ 。前向传播时记录每个池化窗口内最大值的位置索引；反向传播仅将上游梯度传递给该最大值位置，其余位置梯度为零。当窗口内存在多个相同最大值 (tie) 时，梯度在这些位置间平均分配，保证梯度总量守恒。Max pooling 起到下采样作用，将特征图空间分辨率减半，同时提供局部平移不变性。

3.2 Architecture and Fair Comparison

$$\begin{aligned} & \text{Conv}(1 \rightarrow 4, 3 \times 3, \text{pad}=1) \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2) \\ & \rightarrow \text{Conv}(4 \rightarrow 8, 3 \times 3, \text{pad}=1) \rightarrow \text{ReLU} \rightarrow \text{Flatten} \rightarrow \text{Linear}(1568 \rightarrow 10) \end{aligned}$$

该架构设计的考量如下：第一层卷积 $1 \rightarrow 4$ 通道使用 3×3 小卷积核配合 padding=1 保持空间尺寸不变 (28×28)，随后 2×2 max pooling 将分辨率降至 14×14 ；第二层卷积 $4 \rightarrow 8$ 通道进一步提取高阶特征，输出 $8 \times 14 \times 14 = 1568$ 维展平后接全连接分类器。通道数 $4 \rightarrow 8$ 遵循“空间减半、通道加倍”的经典范式，使各层计算量大致均衡。整个网络仅 16 026 个参数，远少于 MLP 的 25 818 个，但归纳偏置更强。

公平比较设置. 为使对比有意义，二者共享数据划分、随机种子、batch size (128)、训练 epochs (5)，均为固定学习率 SGD（无 momentum/dropout/scheduler）。学习率通过预试验分别选定：MLP $\eta=0.1$ ，CNN $\eta=0.05$ 。CNN 学习率较低是因为卷积权重共享使有效梯度更大，需要较小步长以保持训练稳定。

Model	Parameters	Best val acc	Test acc	Test errors
MLP baseline	25 818	0.9479	0.9544	456
CNN baseline	16 026	0.9757	0.9779	221
Δ (CNN – MLP)		+0.0278	+0.0235	–51.5%

表 4: MLP vs. CNN. CNN achieves higher accuracy with 38% fewer parameters.

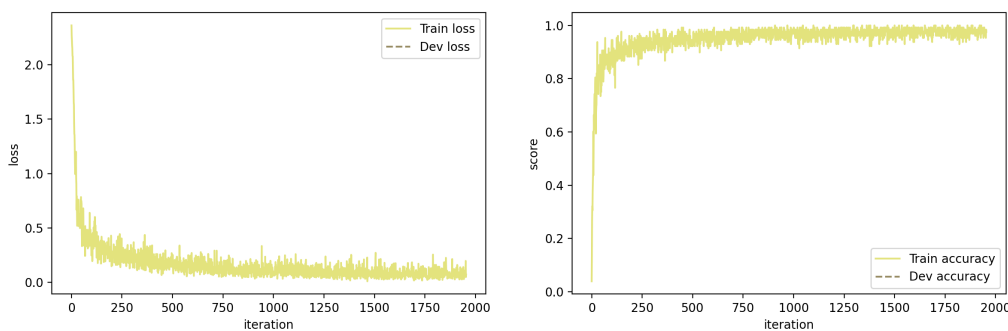


图 2: CNN baseline learning curve.

CNN 优于 MLP 的原因分析. 从表 4 可见, CNN 在参数量减少 38% 的情况下将错误数从 456 降至 221 (减少 51.5%), 这一显著提升源于卷积网络的三个核心归纳偏置:

1. **局部连接 (Local connectivity)**. 每个卷积神经元仅关注 3×3 的局部感受野, 天然适配手写数字中笔画、边缘、拐角等局部特征的检测。MLP 的全连接层则需要从所有 784 个像素中学习相关性, 信噪比更低。
2. **权重共享 (Weight sharing)**. 同一卷积核在所有空间位置复用, 意味着一个学到的边缘检测器可以在图像任何位置发挥作用。这不仅大幅减少参数量 (如第一层卷积仅 $4 \times 1 \times 3 \times 3 + 4 = 40$ 个参数, 而 MLP 第一层为 $784 \times 32 + 32 = 25120$ 个), 还使特征提取具有平移等变性。
3. **池化下采样 (Pooling)**. Max pooling 在 2×2 窗口内取最大值, 使网络对小幅度平移 (± 1 像素) 具有不变性, 增强了对手写体位置变化的鲁棒性, 同时逐步扩大高层神经元的有效感受野。

4 Part C Direction 1: Optimization

在固定 CNN 架构 (Part B) 前提下研究优化策略对收敛速度和最终精度的影响。实验遵循“每次只改变一个主要因素”的控制变量原则, 确保观察到的差异可归因于所改变的因素。

SGD (随机梯度下降):

$$\theta_{t+1} = \theta_t - \eta \nabla L_t.$$

最基础的优化器, 每步沿当前 mini-batch 梯度方向更新参数。优点是实现简单、内存开销小; 缺点是在损失曲面的狭长谷地中容易振荡, 且对所有参数使用统一学习率, 无法自适应调节。

Momentum SGD ($\mu=0.9$):

$$v_{t+1} = \mu v_t - \eta \nabla L_t, \quad \theta_{t+1} = \theta_t + v_{t+1}.$$

引入动量项 v_t 累积历史梯度的指数加权平均。物理直觉为: 参数更新如同带有惯性的小球在损失曲面上滚动——在梯度方向一致的维度上加速, 在振荡方向上相互抵消。动量系数 $\mu = 0.9$ 意味着有效窗口约为最近 $1/(1-\mu) = 10$ 步的梯度。这使得优化器能更快穿越平坦区域并抑制噪声梯度的振荡。

Adam ($\beta_1=0.9, \beta_2=0.999, \epsilon=10^{-8}$): 同时维护梯度的一阶矩估计 m_t (均值) 和二阶矩估计 v_t (未中心方差), 并进行偏差校正以补偿初始化为零带来的低估:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla L_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) (\nabla L_t)^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}.$$

Adam 为每个参数自适应调整有效学习率：梯度方差大的参数步长自动缩小，反之增大。这使其对学习率初始值不太敏感，通常收敛更快。但在小模型、少 epoch 的场景下，其自适应性优势可能不如 Momentum 的简洁高效。

StepLR 学习率调度：每隔 T_{step} 个 epoch 将学习率乘以衰减因子 γ ： $\eta_t = \eta_0 \cdot \gamma^{\lfloor t/T_{\text{step}} \rfloor}$ 。学习率衰减的动机是：训练初期使用较大步长快速逼近最优区域，后期减小步长以细化收敛、减少在最优解附近的振荡。但 milestone 设置不当可能导致过早衰减，使模型在充分训练前就陷入低学习率区间。

5 Part C Direction 2: Regularization

研究 weight decay 和 dropout 两种正则化手段对泛化性能的影响。正则化的核心思想是通过限制模型复杂度来缩小训练误差与测试误差之间的泛化 gap。单项实验以 Momentum CNN 为更强的 baseline（而非原始 SGD CNN），只改变一个因素，以便观察正则化的边际增益。

Weight decay ($\lambda=10^{-4}$): 在每次参数更新时对权重施加衰减： $W \leftarrow (1-\eta\lambda)W - \eta\nabla L_W$ 。对于标准 SGD，这等价于在损失函数中添加 L2 正则项 $\frac{\lambda}{2}\|W\|_2^2$ ，其梯度 λW 与权重衰减项数学等价。但对于 Momentum 和 Adam 等使用梯度历史的优化器，L2 正则化将惩罚项混入梯度矩估计，与直接对权重做衰减（decoupled weight decay）效果不同。本实现采用 decoupled 方式，仅对权重矩阵 W 施加衰减，不对偏置 b 施加，因为偏置参数量少且不影响模型复杂度。Weight decay 鼓励权重取较小值，倾向于更平滑的决策边界，从而减少过拟合。

Dropout ($p=0.2$): 训练时在全连接层前，以概率 p 将每个神经元的输出随机置零，并将保留的激活值除以 $(1-p)$ （inverted dropout），使训练和测试时的期望激活值保持一致。测试时不做任何操作（恒等映射）。Dropout 的正则化效果可以从两个角度理解：(1) 它迫使网络不能依赖任何单个神经元，从而学习更分散、更鲁棒的特征表示；(2) 它近似于对指数多个“子网络”做模型平均（ensemble），每个子网络对应一种随机 mask，测试时的确定性前向传播隐式聚合了这些子网络的预测。

Early stopping: 在每个 epoch 结束后评估 validation accuracy，若连续若干 epoch 无提升则提前终止训练，使用验证集上表现最好的模型参数。Early stopping 本质上是一种隐式正则化：它限制了优化过程的有效迭代次数，防止模型在训练集上过度拟合。在本实验 5 epoch 的设置下，early stopping 未被实际触发（模型仍在持续改善），但将其作为安全机制纳入组合 recipe，以防更长训练中出现过拟合。

6 Part C Results

Recipe	Optimizer	Scheduler	Weight decay	Dropout	Early stop
sgd	SGD	—	0	0	no
momentum	Momentum	—	0	0	no
sgd_step	SGD	StepLR	0	0	no
adam	Adam	—	0	0	no
l2	Momentum	—	10^{-4}	0	no
dropout	Momentum	—	0	0.2	no
recipe_combo	Momentum	MultiStepLR	10^{-4}	0.1	yes

表 5: Recipe configurations. All share the same CNN, data split, seed, batch size (128), epochs (5).

Recipe	Best val acc	Test acc	Δ test acc	Final dev loss
sgd	0.9757	0.9779	—	0.0870
momentum	0.9821	0.9836	+0.0057	0.0644
sgd_step	0.9732	0.9739	−0.0040	0.0942
adam	0.9776	0.9798	+0.0019	0.0774
l2	0.9820	0.9836	+0.0057	0.0647
dropout	0.9839	0.9837	+0.0058	0.0727
recipe_combo	0.9840	0.9856	+0.0077	0.0548

表 6: Part C results. Δ is relative to the sgd baseline.

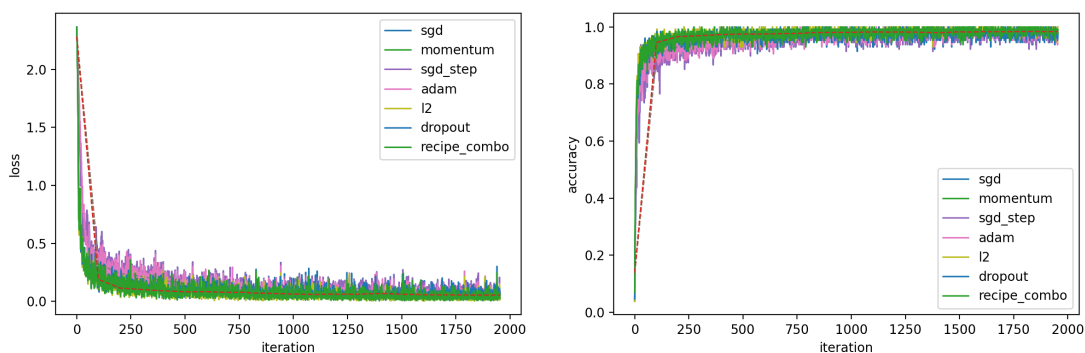


图 3: Learning curves of all recipes (solid: training, dashed: validation).

Analysis

Momentum 是最显著的单项提升来源: test acc 从 0.9779 提升至 0.9836 (+0.0057)。从学习曲线 (图 3) 可以看到, Momentum 的训练 loss 在前 1–2 个 epoch 内下降速度明显快于 SGD, 且最终收敛到更低的 loss 值。这是因为动量累积使梯度更新更稳定、更有方向性, 尤其在损失曲面存在各向异性 (不同方向曲率差异大) 时效果显著。

Adam 优于 SGD (+0.0019) 但不如 Momentum (−0.0038)。Adam 的自适应学习率在大型模型和长训练中优势明显, 但对于当前的小型 CNN (16 026 参数) 和短训练 (5 epochs), 固定动量已足够高效。Adam 的额外复杂性 (维护 m_t 、 v_t 两组状态) 未带来对等收益, 且其自适应缩放在某些方向上可能过于保守。

StepLR 反而造成性能下降 (-0.0040)。在当前设置下, StepLR 的 milestone 在约 epoch 2 处触发, 学习率从 0.05 骤降至 0.0125 (衰减因子 $\gamma = 0.25$)。此时模型尚未充分收敛, 过早的大幅衰减导致后续 3 个 epoch 的有效学习能力不足。这一结果说明学习率调度器高度依赖超参数选择, 需与总训练 epoch 数、模型收敛速度匹配, 盲目使用可能适得其反。

Weight decay 在 Momentum baseline 上几乎无额外增益 (test acc 同为 0.9836)。这表明当前小型 CNN 在 5 epoch 的 MNIST 训练中过拟合并不严重——从学习曲线可见训练 loss 与验证 loss 的 gap 较小, 正则化无显著发力空间。

Dropout 取得单项实验中最高的 val acc (0.9839), 但 test acc (0.9837) 与 Momentum 基本持平。Dropout 略微增大了训练 loss (因为训练时随机丢弃神经元), 但同时缩小了 train-val gap, 表现出正则化效果。其 final dev loss (0.0727) 高于 Momentum (0.0644), 说明 Dropout 牺牲了一定的训练拟合度以换取泛化。

Recipe combo (Momentum + MultiStepLR + Weight decay 10^{-4} + Dropout 0.1 + Early stopping) 达到最佳 test acc 0.9856, 相比 SGD baseline 提升 $+0.0077$ 。组合方案同时利用了 Momentum 的快速收敛、MultiStepLR 的精细调节、以及轻度正则化的泛化增益。但由于同时改变了多个因素, 其 $+0.0020$ 的额外提升 (相对 Momentum 单项) 无法精确归因于某一组件, 应理解为整体协同效果。

7 Visualization and Error Analysis

7.1 Confusion Matrix and Misclassified Samples

Model	Test acc	Errors	Top confusions
MLP	0.9544	456	9 $\rightarrow$ 4, 5 $\rightarrow$ 3, 7 $\rightarrow$ 9, 8 $\rightarrow$ 3
CNN	0.9779	221	7 $\rightarrow$ 2, 9 $\rightarrow$ 4, 6 $\rightarrow$ 5, 2 $\rightarrow$ 1
Best recipe	0.9856	144	4 $\rightarrow$ 9, 5 $\rightarrow$ 3, 7 $\rightarrow$ 2, 8 $\rightarrow$ 0

表 7: Error counts and top confusion pairs across three models.

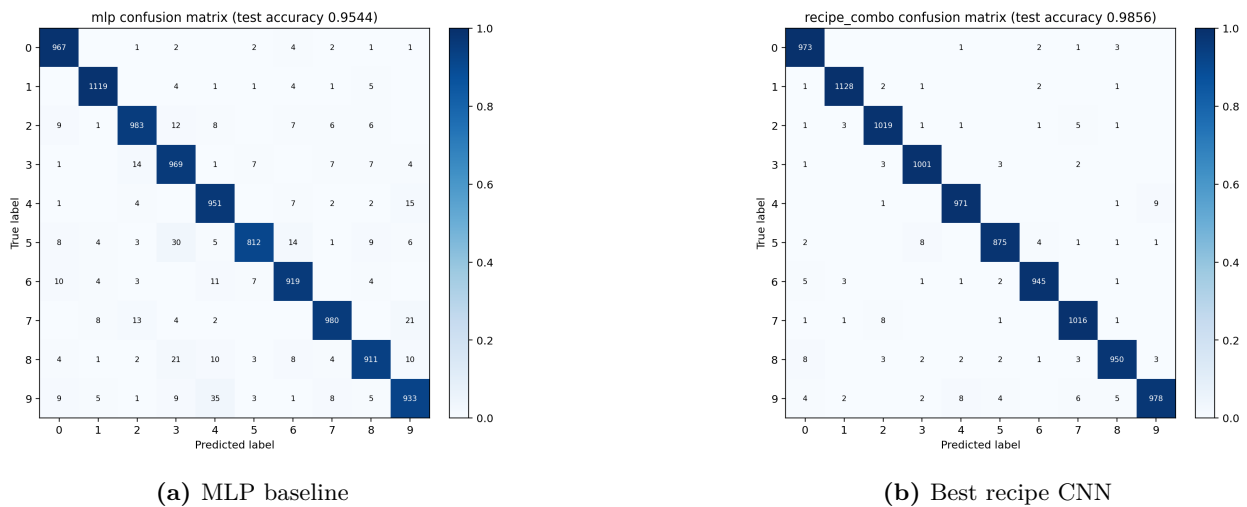


图 4: Confusion matrices on the test set.

从混淆矩阵和 top confusion 列表（表 7）可以看出，三个模型的错误分布有以下规律：

- **错误总量随模型能力递减：** MLP 456 $\rightarrow$ CNN 221 $\rightarrow$ Best recipe 144，每一步都接近减半。其中 MLP $\rightarrow$ CNN 的下降主要由结构改进（卷积归纳偏置）贡献，CNN $\rightarrow$ Best recipe 的下降则由优化与正则化（Momentum + L2 + Dropout）贡献。
- **易混淆对集中在视觉相似的数字之间：** 4 $\leftrightarrow$ 9（上方闭环是否封闭）、5 $\leftrightarrow$ 3（左侧弧线方向）、7 $\leftrightarrow$ 2（底部是否带横线）、8 $\leftrightarrow$ 0（中部是否有交叉）。这些混淆对在三个模型中反复出现，说明它们反映了 MNIST 数据本身的固有难度，而非某个模型的特定弱点。
- **MLP 特有的混淆模式（如 9 $\rightarrow$ 4, 8 $\rightarrow$ 3）** 在 CNN 中显著缓解，说明卷积结构确实改善了对局部笔画结构的判别。

观察图 5 中的高置信度错分样本可以发现，绝大多数残留错误并非模型缺陷，而是源于真实的视觉歧义：闭环不完整使 9 看起来像 4、起笔位置异常导致 5 与 3 难以区分、笔画断裂使 8 失去拓扑特征。在这些样本上，即使人类标注者也可能产生分歧。

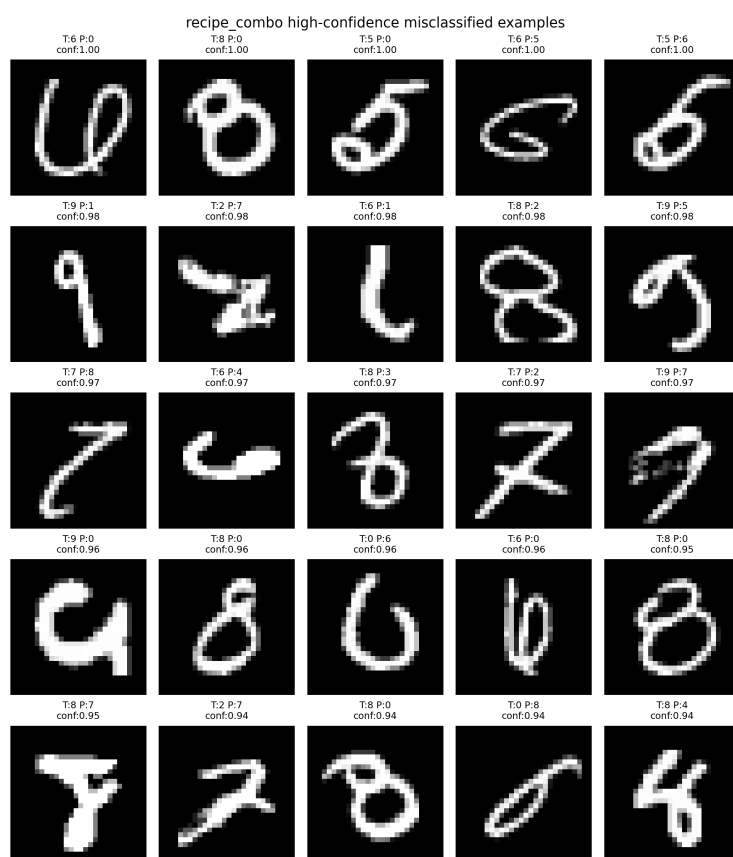


图 5: Best recipe CNN: high-confidence misclassified examples.

7.2 Weights and Kernels

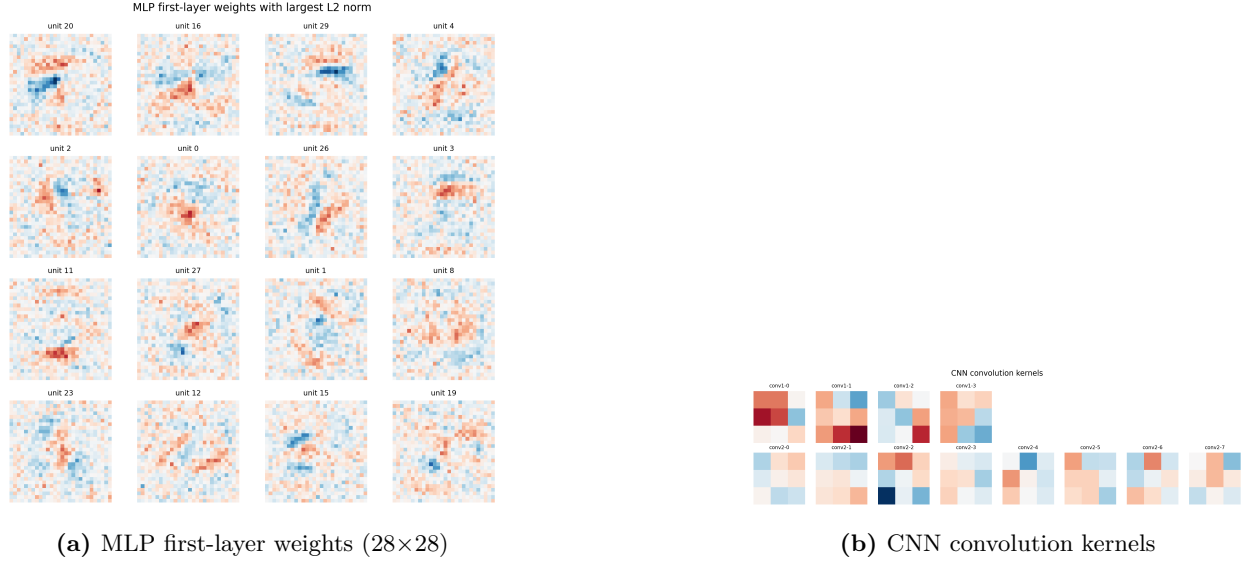


图 6: MLP 权重是全图模板; CNN 核仅 3×3 , 为局部边缘/方向检测器。

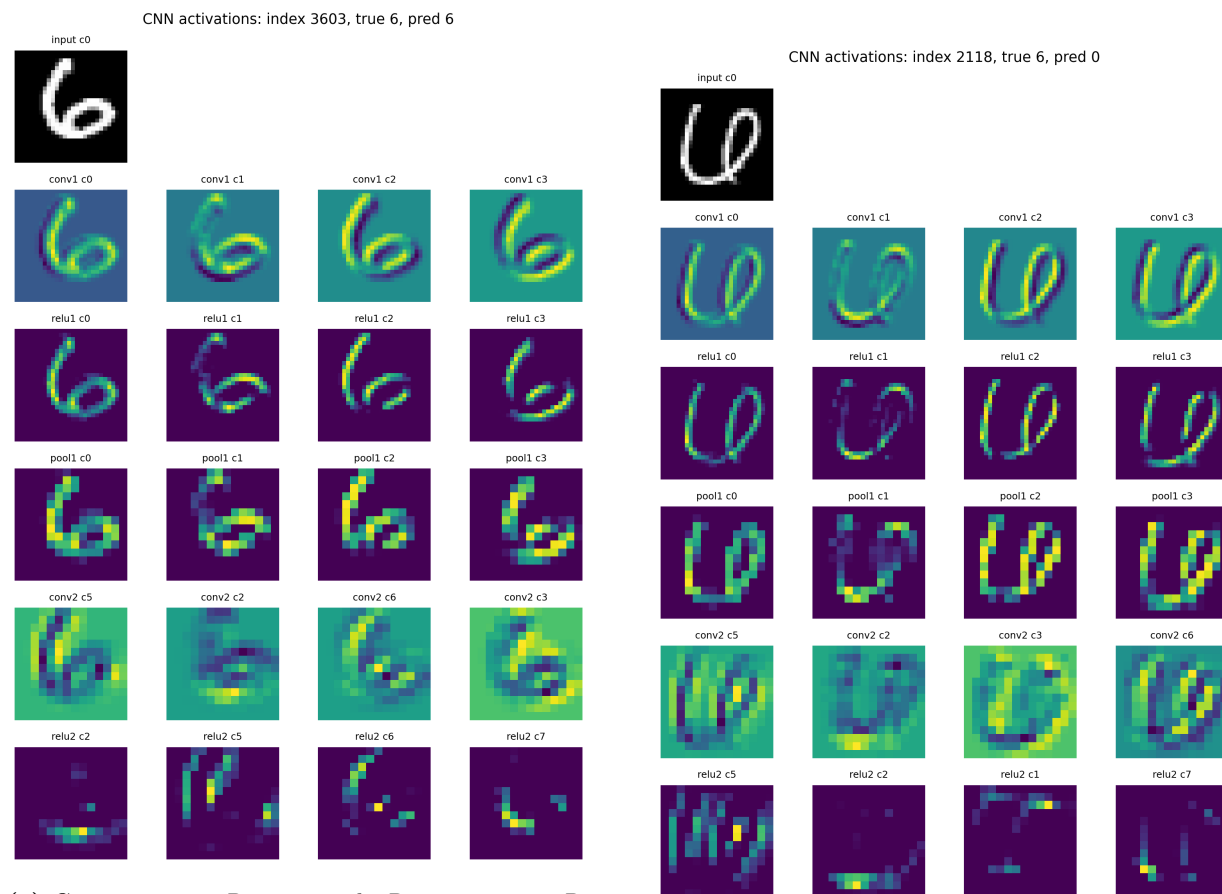
可视化结果鲜明地反映了两类模型的内在差异。MLP 第一层的每个神经元权重展开为 28×28 后, 呈现出与某个数字模糊轮廓相似的“全局模板” (template matching) 形式: 网络通过将输入与这些模板做加权内积来判断相似度。这种表示方式空间利用率低, 且对位置/姿态变化敏感。相比之下, CNN 的 3×3 卷积核虽然参数极少, 但每个核都明显呈现出方向性边缘检测器、对角线检测器或斑点检测器的形态, 与经典图像处理中的 Sobel/Prewitt 滤波器具有异曲同工之妙。这印证了 CNN 通过组合局部基元、在空间上反复使用同一检测器来构建分层特征的设计哲学。

7.3 Intermediate Feature Maps

为深入理解 CNN 的分层特征提取过程, 选取一个正确分类样本 (数字 6 被正确识别为 6) 和一个错分样本 (数字 6 被误判为 0), 逐层记录中间 feature maps 的激活情况。为定量度量特征图对笔画区域的关注程度, 定义前景/背景激活比 R :

$$R = \frac{\text{mean}_{\text{foreground}}}{\text{mean}_{\text{background}}},$$

其中前景定义为原始图像中像素值大于阈值的区域 (数字笔画位置), 背景为其余区域。 R 越大说明特征图越聚焦于笔画区域; R 接近 1 则说明特征图对前景/背景的反应趋于均匀, 即特征已不再是简单的边缘描绘, 而是更抽象的组合表示。



(a) Correct: conv1 $R \approx 9.6$, pool1 $R \approx 11.5$, conv2 $R \approx 2.9$.

(b) $6 \rightarrow 0$: conv2 $R \approx 2.0$, 组合特征误解为 0.

图 7: CNN intermediate activations (digit 6). 浅层强化笔画区域 (高 R); 深层 R 降低, 第二层在更抽象层面组合 stroke patterns 而非简单描边。

从 R 值的变化可以清晰看出 CNN 的分层抽象过程。在正确样本上, conv1 阶段 $R \approx 9.6$, pool1 阶段 $R \approx 11.5$ (pooling 进一步强化了最显著响应), 表明浅层卷积核确实在执行类似边缘描绘的任务, 将笔画区域显著高亮; 进入 conv2 后 R 降至 ≈ 2.9 , 前景/背景的对比度大幅减小, 说明第二层不再简单地“勾勒笔画”, 而是开始组合空间上分散的 stroke patterns (如“闭合环”、“下方弧”等组合特征), 这些抽象特征在背景区域也可能产生有意义的响应。

在错分样本 ($6 \rightarrow 0$) 上, conv2 的 $R \approx 2.0$, 进一步降低, 说明此样本在第二层的特征组合阶段被误解为 0 的典型模式 (闭合环 + 对称形状)。这一观察支持了一个常见的误解需要修正: “CNN 提取的是图像轮廓”——实际上只有浅层卷积近似边缘检测, 深层网络已脱离像素层面, 在抽象的特征空间中进行模式组合与判别。

8 Main Results

Setting	Val acc	Test acc	Note
MLP baseline	0.9479	0.9544	Part A, 25 818 params
CNN baseline	0.9757	0.9779	Part B, 16 026 params
CNN + Momentum	0.9821	0.9836	Strongest single-factor gain
CNN + Adam	0.9776	0.9798	Better than SGD, < Momentum
CNN + Weight decay	0.9820	0.9836	$\approx$ Momentum
CNN + Dropout	0.9839	0.9837	Best single-factor val acc
Best recipe	0.9840	0.9856	Momentum+StepLR+L2+dropout

表 8: Summary of all experimental results.

9 Discussion

- 结构归纳偏置 vs. 暴力拟合.** CNN 用更少参数 (16 026 vs 25 818) 获得更高精度 (0.9779 vs 0.9544), 错误数减少 51.5%。这一对比有力地说明: 在视觉任务中, 恰当的结构先验 (局部连接、权重共享、平移不变性) 比单纯增加参数更有效。MLP 必须从数据中学习“像素邻接”、“特征可平移”等基本事实, 而 CNN 将这些先验直接编码到架构中, 从而把模型容量集中在真正需要学习的判别特征上。这一观察与“no free lunch”定理一致——在特定任务结构上有效的归纳偏置才是模型设计的核心。
- 优化器选择.** Momentum 是当前实验中最有效的单项改进 (+0.0057), 既加快收敛速度又提升最终精度。Adam 虽具备自适应学习率的理论优势, 但在小模型 + 短训练的设置下未能超过 Momentum, 说明“更复杂的优化器”并不总是“更好的优化器”。StepLR 在当前设置下反而有害 (-0.0040), 凸显学习率调度器需要与训练总长度、模型收敛速度精细匹配——盲目使用反而损害性能。这提示在资源受限的场景下, 调参成本本身也是优化器选择的一个隐性维度。
- 正则化的边际效用.** Weight decay 和 dropout 在 Momentum baseline 之上的收益温和 ($\leq +0.0001$ test acc)。这并非正则化无效, 而是反映了当前任务的过拟合压力较小: 模型容量小 (16 026 参数)、数据相对充足 (50 000 训练样本)、训练 epoch 数有限 (5)。在更大的模型或更长的训练中, 正则化的作用会显著增强。组合 recipe 达到最佳 0.9856, 但同时改变多个因素, 其额外收益 (+0.0020 over Momentum) 应理解为整体协同效果, 无法精确归因到单一组件。
- 错误本质与改进上限.** 残留 144 个错误中绝大多数集中在 $4 \leftrightarrow 9$ 、 $5 \leftrightarrow 3$ 、 $7 \leftrightarrow 2$ 、 $8 \leftrightarrow 0$ 等视觉歧义对上, 且高置信度错分样本本身存在客观的标注模糊性。这意味着: 在不引入额外监督信号 (如更精细的标签、数据增强、外部知识) 的情况下, 仅依靠优化技巧的提升空间已接近极限, 进一步的精度突破需要从数据与架构层面入手。
- 对 CNN 工作机制的修正.** feature map 可视化显示 CNN 并非全程“提取轮廓”。仅浅层卷积 (conv1) 的激活近似边缘检测 ($R \approx 9.6$), 深层 (conv2, $R \approx 2.9$) 已脱离像素描绘, 在抽象特征空间中组合 stroke patterns。这与“CNN 是层级抽象器”的现代理解一致, 纠正了“卷积 = 边缘检测”的过度简化。
- 实现可靠性.** 所有算子 (Linear、conv2D、ReLU、MaxPool2D、Softmax CE) 基于 NumPy 从零实现, 通过中心差分法的数值梯度检验全部通过 (相对误差 $< 10^{-8}$), 保证了反向传播的数学正确性。完整代码与 checkpoint 可从 GitHub/ModelScope 一键复现。这一可重复性是后

续所有比较实验结论可信的前提。